CMC UNIVERSITY

Aspire to Inspire the Digital World

Machine Learning and Data Mining

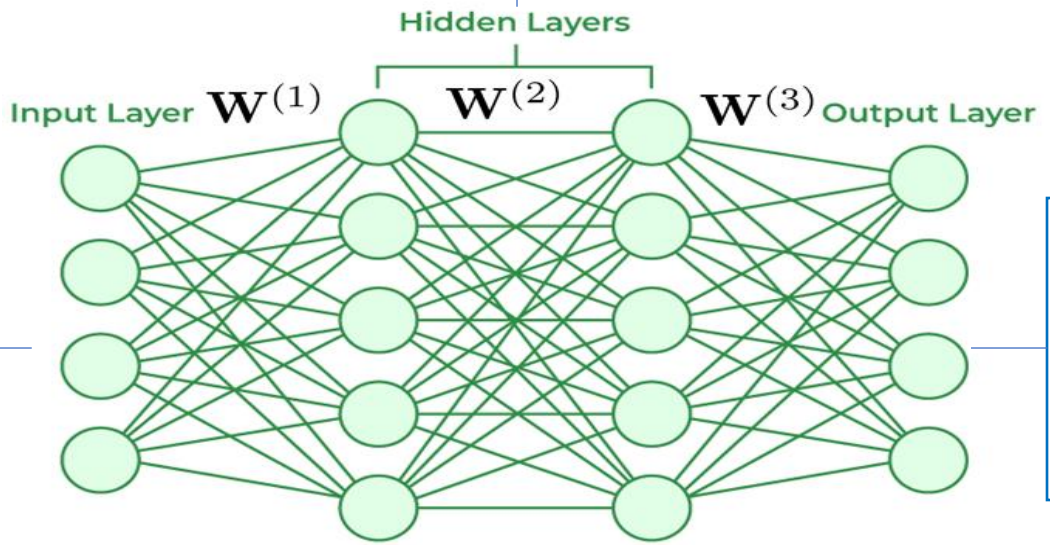
Ha Noi, 2025

LESSON 1.2
Neural Network (continues)

Multi-layer neural network

Input Layer: Receives the initial data and feeds the data features into the neural network. Performs no complex calculations, merely forwarding data to another layer.

Hidden Layers: Perform the learning process by extracting data features ranging from simple to complex.



Output Layer: Synthesizes the learned information to produce the final result, such as predicting a value or classifying data.

Number of neurons per layer	Input: 4	Hidden 1: 5	Hidden 2: 5	Output: 4
Connections	Each connection has a weight and a bias.			
Data flow direction	Left → Right			
Goal	Learn useful features to accurately predict results.			

Multi-layer neural network

Advantages of multilayer neural networks

Learns complex non-linear functions

The model can capture non-linear relationships in the data.

Automatic feature extraction

Hidden layers automatically find important features.

Wide applications

Image recognition, natural language processing, prediction, classification, recommendation,...

Flexible and scalable

Can add or remove hidden layers or neurons depending on the complexity of the problem.

Multilayer neural networks are the foundation of modern Deep Learning. Clearly understanding the roles of the 3 types of layers helps us build more effective models!

Example of a neural network for classification

Given a dataset with 5 input features (x_1, x_2, x_3, x_4, x_5) and 3 output classes

Feature x_1	Feature x_2	Feature x_3	Feature x_4	Feature x_5	Label
1	22	569	35	0	Class 1
1	7	351	75	1	Class 2
1	45	451	542	1	Class 2
1	5	572	8	0	Class 1
0	22	565	44	1	Class 3
0	24	243	546	1	Class 3
1	78	953	42	0	Class 2

Example of a neural network for classification

Input Layer: 5 neurons

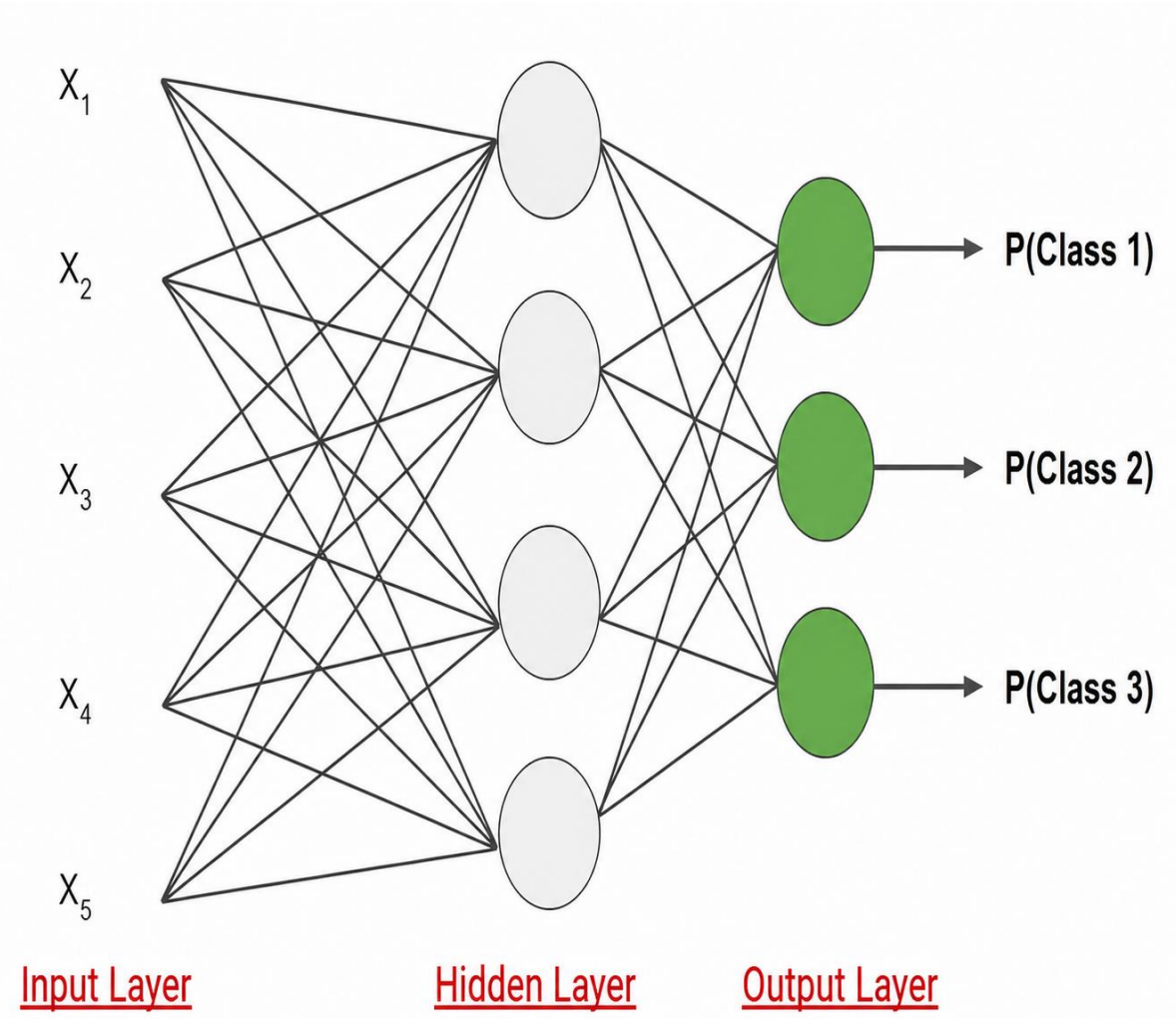
Fixed. Equal to the number of features from x_1 đến x_5.

Hidden Layer: 4 neurons

Flexible hyperparameter. Determined by the engineer based on the complexity of the problem.

Hidden Layer: 3 neurons

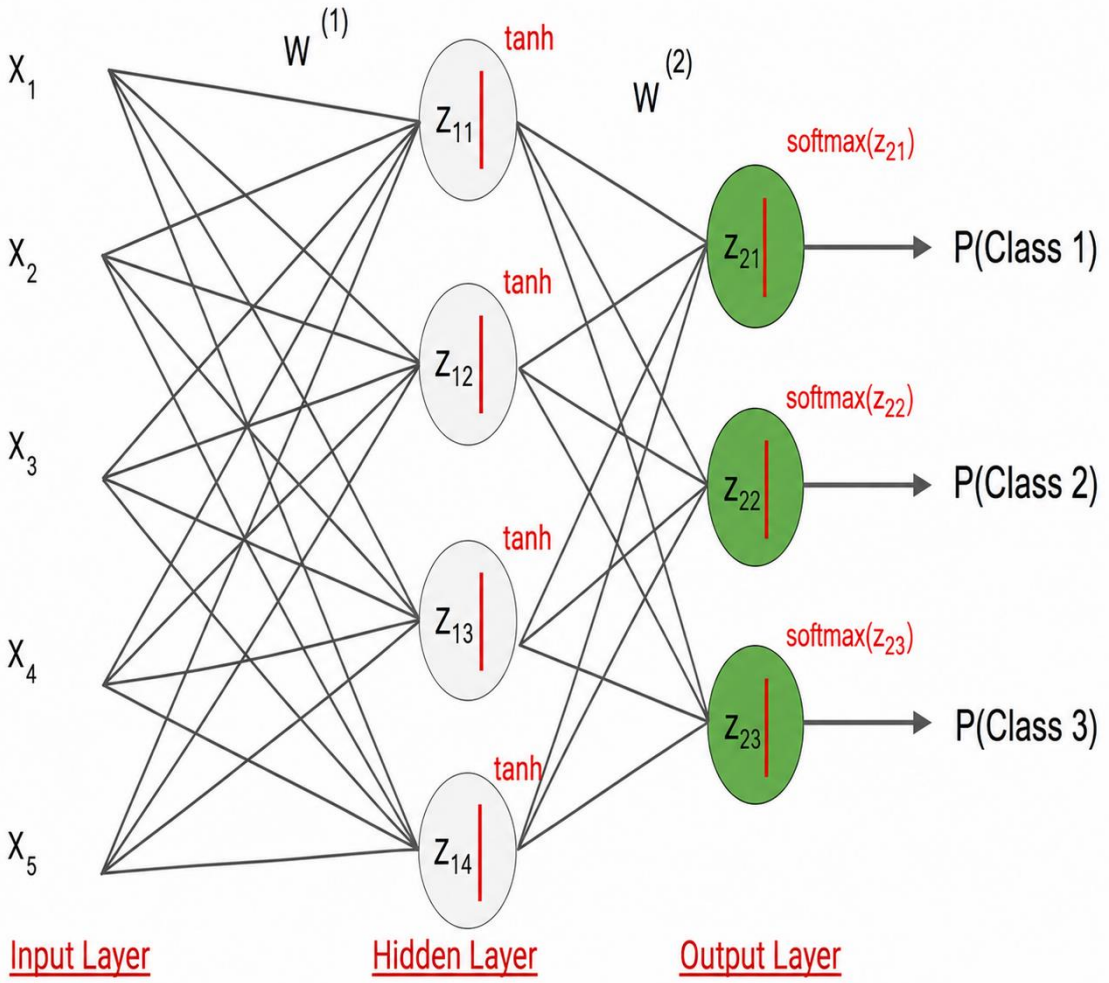
Fixed. Exactly equal to the number of classes (Class 1, Class 2, Class 3).



Example of a neural network for classification

Each hidden layer neuron uses inputs, weights, and biases to calculate a sum (denoted as z_{11}, z_{12}, z_{13}, z_{14}) with the formula:

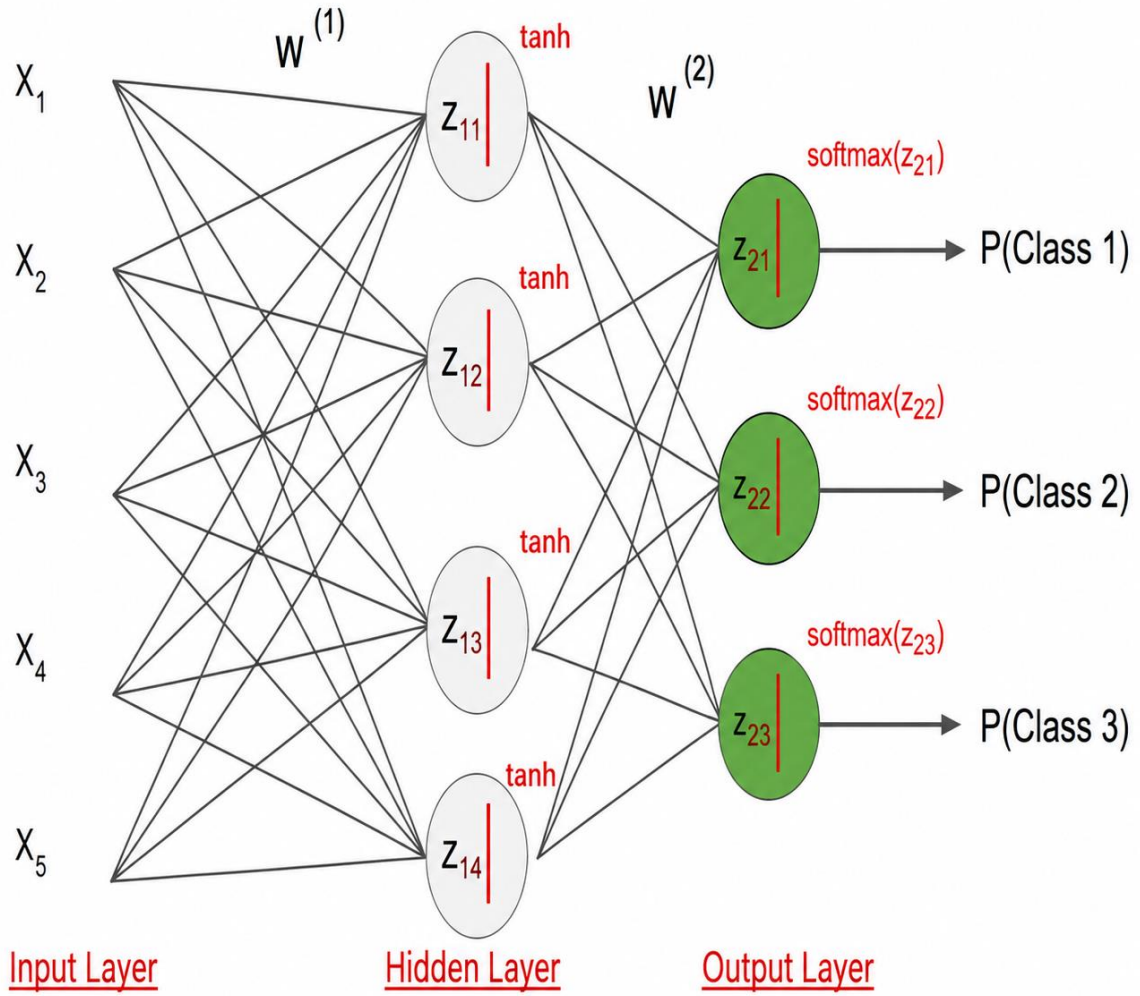
$$z_{1j} = \sum (x_i * w_{ij}) + \text{bias}$$



Example of a neural network for classification

For each $z_{1j,}$ each value will pass through the Tanh function. The outputs of the value layers mapped through the Tanh function are: $Tanh(z_{11})$, $Tanh(z_{12})$, $Tanh(z_{13})$, $Tanh(z_{14})$

These values $Tanh(z_{11})$, $Tanh(z_{12})$, $Tanh(z_{13})$, $Tanh(z_{14})$ will act as the new input vector for the next layer..

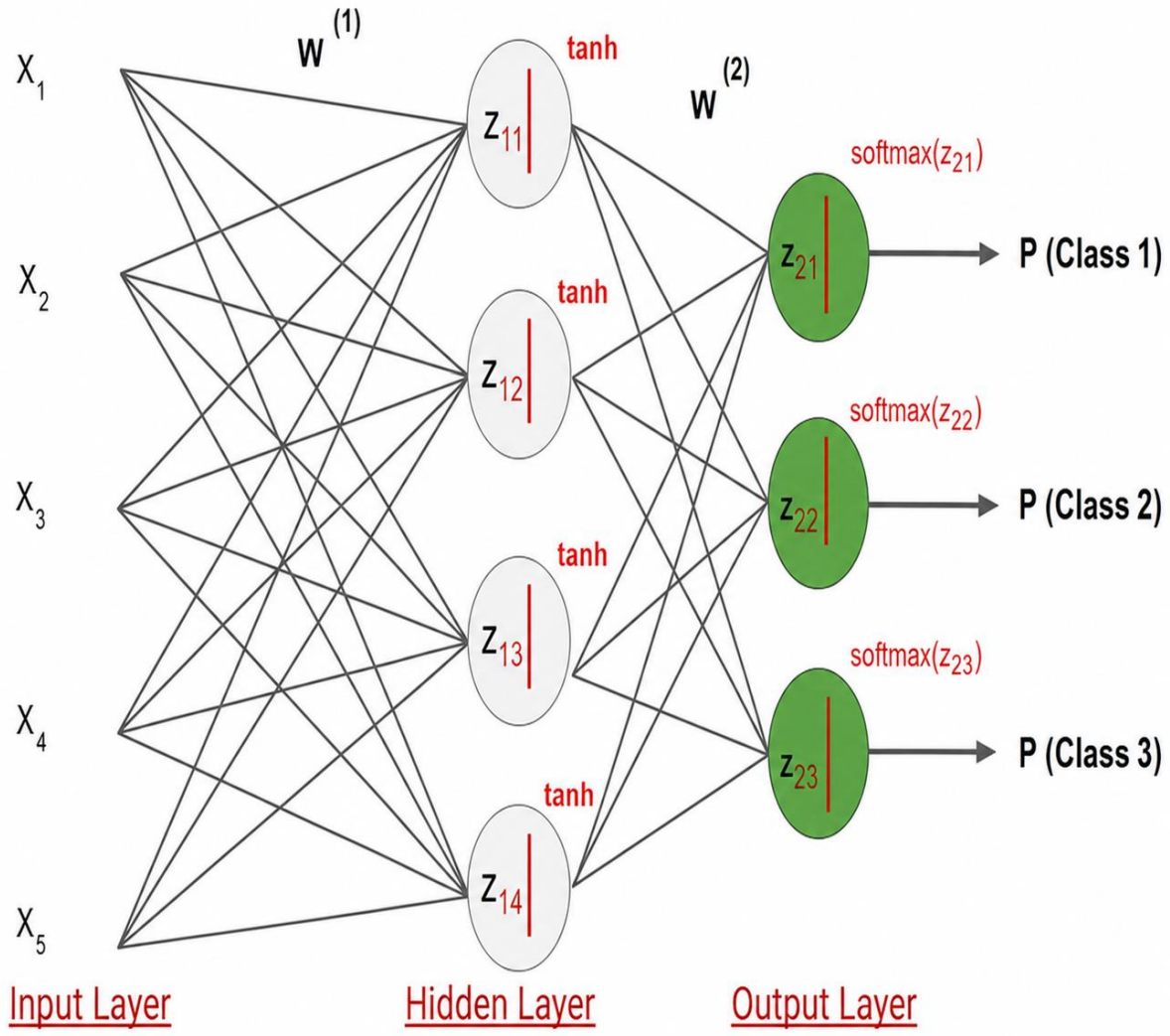


Example of a neural network for classification

$$Z_{21} = w_{11}^{(2)}\mathrm{Tanh}(z_{11}) + w_{21}^{(2)}\mathrm{Tanh}(z_{12}) + w_{31}^{(2)}\mathrm{Tanh}(z_{13}) + w_{41}^{(2)}\mathrm{Tanh}(z_{14})$$

$$Z_{22} = w_{12}^{(2)}\mathrm{Tanh}(z_{11}) + w_{22}^{(2)}\mathrm{Tanh}(z_{12}) + w_{32}^{(2)}\mathrm{Tanh}(z_{13}) + w_{42}^{(2)}\mathrm{Tanh}(z_{14})$$

$$Z_{23} = w_{13}^{(2)}\mathrm{Tanh}(z_{11}) + w_{23}^{(2)}\mathrm{Tanh}(z_{12}) + w_{33}^{(2)}\mathrm{Tanh}(z_{13}) + w_{43}^{(2)}\mathrm{Tanh}(z_{14})$$



Example of a neural network for classification

CMC UNIVERSITY
Aspire to Inspire the Digital World

Calculate Softmax(Z_{21}), Softmax(Z_{22}), and Softmax(Z_{23})

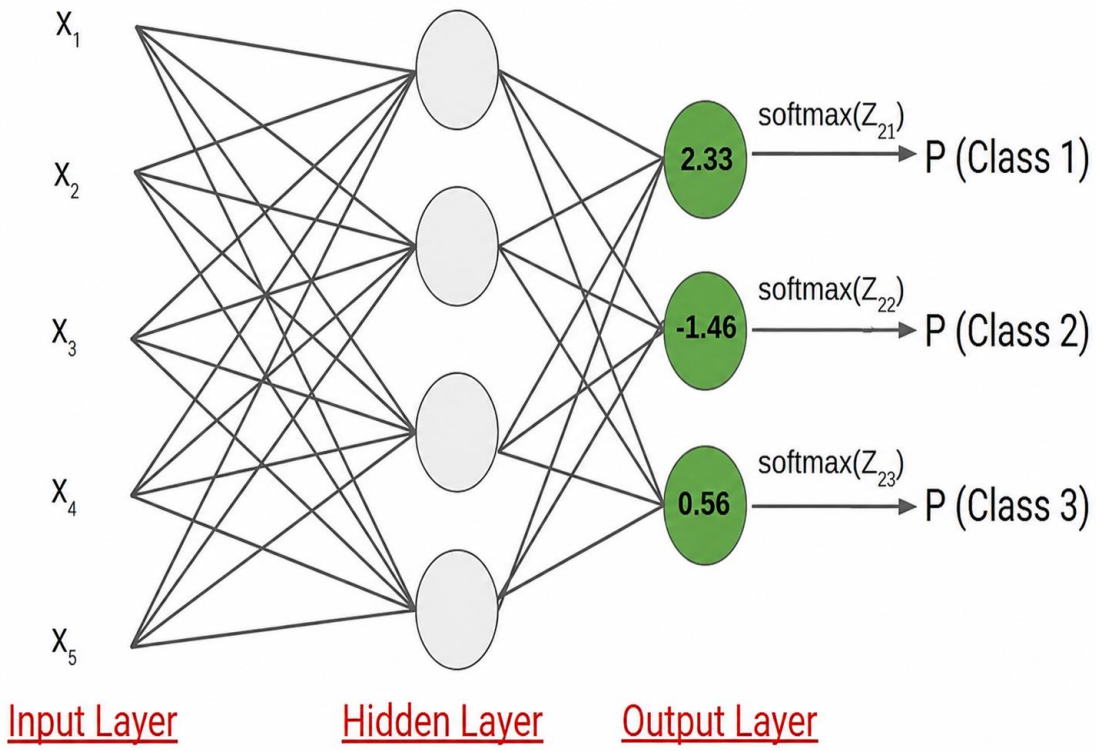
Logits: $Z_{21} = 2.33, Z_{22} = -1.46, Z_{23} = 0.56$

$$softmax(z_{21}) = \frac{\exp(z_{21})}{\exp(z_{21}) + \exp(z_{22}) + \exp(z_{23})}$$

z_{21} = 2.33 → **P(Class 1)** = $\frac{\exp(2.33)}{\exp(2.33) + \exp(-1.46) + \exp(0.56)}$ = 0.83827314

z_{22} = -1.46 → **P(Class 2)** = $\frac{\exp(-1.46)}{\exp(2.33) + \exp(-1.46) + \exp(0.56)}$ = 0.01894129

z_{23} = 0.56 → **P(Class 3)** = $\frac{\exp(0.56)}{\exp(2.33) + \exp(-1.46) + \exp(0.56)}$ = 0.14278557



How to implement a neural network

	Feedforward networks	Recurrent networks	Convolutional networks
Input Data	Tabular data, static figures	Sequence data, text, audio signals	Images, video
Complexity	Basic, easy to train and deploy	High. Captures sequential dependencies	High. Extracts spatial features
Applications	Basic prediction and classification	Natural Language Processing (NLP), time series	Computer Vision, object recognition

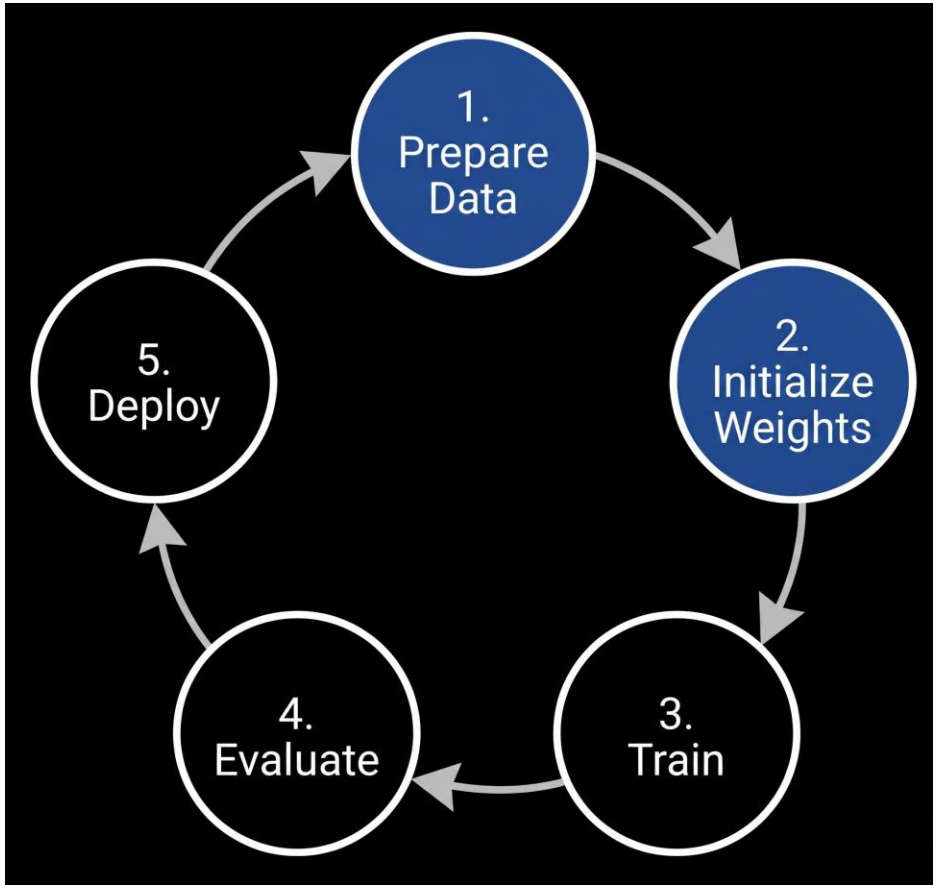
How to implement a neural network

Step 1: Data Preparation

- Collect and preprocess data to ensure it fits the required format for the neural network.
- Data preprocessing includes normalization, scaling, or data augmentation to improve network performance and generalization capability.

Step 2: Initialize Weights and Biases

- Randomly initialize the weights and biases of the neural network.
- Proper initialization is crucial for efficient training, as it can significantly impact convergence speed and the network's final performance.



How to implement a neural network

Step 3: Define the Loss Function

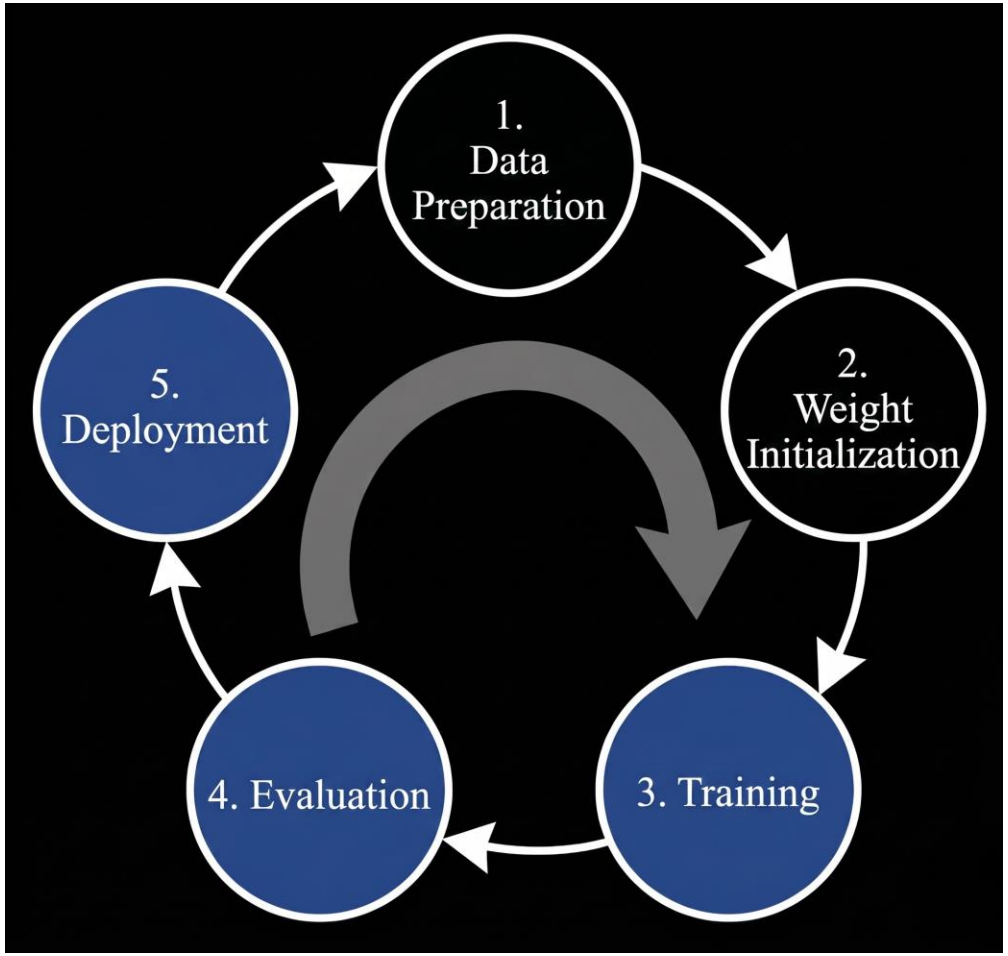
- Measure discrepancy: Use MSE for regression tasks or Cross-Entropy for classification problems.

Step 4: Training (Optimization)

- Utilize optimization algorithms: Use Gradient Descent, Adam, or RMSprop.The system automatically adjusts the weights to minimize the loss function.

Step 5: Validation

- Evaluate on a separate set: Assess the model's performance on a distinct Validation set. Adjust settings such as Learning rate, Batch size, and the number of hidden layers to find the optimal configuration.



Challenges with neural networks

Data Requirements

- Neural networks require a large amount of labeled data for training, which can be challenging to collect, label, and preprocess.
- Poor quality or insufficient data can lead to overfitting or suboptimal performance.

Computational Complexity

- Training neural networks, especially deep learning models, requires significant hardware power such as GPUs or TPUs.

Challenges with neural networks

Model Explainability

- Neural networks are often considered "black boxes" due to their complex internal structures, making it difficult to understand how they arrive at specific predictions.

Hyperparameter Tuning

- Selecting the right hyperparameters—such as learning rate, network architecture, and activation functions—can be challenging and time-consuming.
- Choosing ineffective hyperparameters can lead to poor performance or slow convergence during the training process.

Challenges with neural networks

Phenomenon	Cause	Solution
Overfitting: High training performance but low generalization on new data.	The model is too sensitive to noise and non-representative local features.	Apply Dropout, Data Augmentation, or Early Stopping.
Poor Convergence: Loss value saturates early or gets stuck at a high error level.	Complex error space causes the model to get stuck in local minima or saddle points.	Change Optimizers, tune the learning rate, or re-initialize weights.
Vanishing Gradient: Update speed at the initial layers is extremely slow or stops entirely.	Gradients diminish as they backpropagate through very deep network structures.	Replace activation functions with ReLU/Leaky ReLU, or integrate Batch Normalization.